

CPE477/ECG677 – Embedded Security & Machine Learning

Design Assignment 4

DO NOT REMOVE THIS PAGE DURING SUBMISSION:

Name: Chetan Dahal

Group-Partner Name:

Email: dahalc1@unlv.nevada.edu

Github Repository link (root): <https://github.com/autonomize009/autonomize009-Assign>

Youtube Playlist link (root):

<https://www.youtube.com/playlist?list=PLVIuzDVvAe9iHvuCgkgMcRMypPbO1yoZG>

Follow the submission guideline to be awarded points for this Assignment.

Submit the following for all Assignments:

1. In the document, for each task submit the modified or included code (from the base code) with highlights and justifications of the modifications. Also include the comments. If no base code is provided, submit the base code for the first task only.
2. Create a private Github repository with a random name (no CPE477/677, Lastname, Firstname). Place all assignments under the root folder, sub-folder named Assignmentn, with one document and one video link file for each lab, place modified c files named as main.c.
3. If multiple 'c' or 'h' files or other libraries are used, place these files inside the folder.
4. The folder should have a) Word document (see template), b) source code file(s) with other 'c' and 'h' include files, c) text file with youtube video links (see template).
5. Submit the pdf file in canvas before the due date. The root folder of the github assignment directory should have the documentation and the text file with youtube video links.
6. Organize your youtube videos as playlist under the name "EMBSEC&ML". The playlist should have the video sequence arranged as submission or due dates.
7. Only submit pdf documents. Do not forget to upload this document in the github repository and in the canvas submission portal.

1. Goal: Explain what is explored in this assignment and what was accomplished?

In this assignment, mainly the use of X-CUBE-AI library was explored. The accelerometer and gyroscope data were read via UART and a 1D CNN model was trained to predict the three arm gym workout motions: bicep curl, hammer curl and wrist rotation. The trained neural network model was uploaded to the MCU using X-CUBE-AI library and the real-time motion classification was obtained.

2. Screenshots of the IDE, physical setup, debugging process – Provide screenshot of successful compilation, screenshots of graphs, etc.

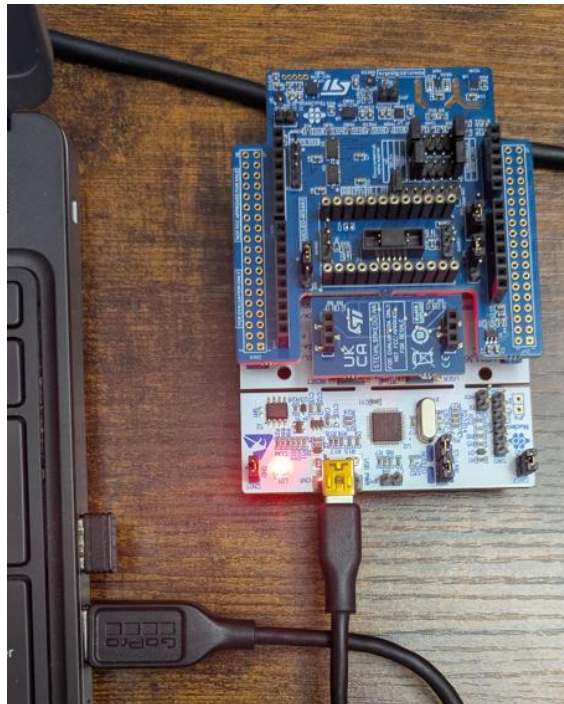


Figure 1: Physical setup of the STM32F401RE and X-NUCLEO-IKS4A1 board

Project Description

This project explores the application of X-CUBE-AI library to classify arm motion using keras-based model. Three gym workout motions of arm were selected for classification. Bicep curl is a weight lifting exercise, where the weight is lifted towards the shoulder by facing the palm towards the shoulder. Hammer curl is slight variation in bicep curl, where the palm faces each other instead of shoulder. The arm motion is same in these two cases but the way in which weight is carried and the palm direction varies. Similarly, wrist rotation is the exercise, in which weight is grabbed in palm and wrist is rotated from right to left and vice-versa, keeping elbow fixed as much as possible.

The motion data for each class were collected separately and stored in separate csv file. The class – 0 represents bicep curl while class – 1 and class – 2 labels denote hammer curl and wrist rotation

respectively. Three accelerometer data and three gyroscope data were recorded as the input feature for training and testing of the neural network model.

Source code description for embedded application

Several header files required for the project compilation were imported. "lsm6dsv16x_reg.h" header was imported as the driver header for IMU to import the sensor's function and talk to it to get the accelerometer and gyroscope samples. "stm32f4xx_hal.h" was imported to access STM32 MCU peripherals like I2C and UART. Similarly, "ai_platform.h", "network.h" and "network_data.h" header files were required to use X-CUBE-AI library, to initialize the neural network model and run inference on them and to use the trained weights and biases for the model.

```
#include <string.h>
#include <stdio.h>
#include "lsm6dsv16x_reg.h"
#include "stm32f4xx_hal.h"
#include "ai_platform.h"
#include "network.h"
#include "network_data.h"
```

Some macros were defined to indicate the sensor bus to be used by the IMU driver. Also, boot time for IMU was defined in the form of macro as seen below.

```
#define SENSOR_BUS hi2c1
#define BOOT_TIME 10 //ms
```

The code below defines the I2C peripheral to use for reading sensor data and the UART for printing classification or sensor output.

```
I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart2;
```

Some arrays to hold accelerometer and gyroscope sensor data and buffer for storing strings to be sent to UART were defined as seen in code below. Similarly, arrays to hold one input window of neural network model, probabilities for each output class and the data for hidden layers were defined. Also, the code below includes the array holding string form of 3 output classes and also defines the pointer to input and output buffer used by X-CUBE-AI.

```
static int16_t data_raw_acceleration[3];
static int16_t data_raw_angular_rate[3];
static double_t acceleration_mg[3];
static double_t angular_rate_mdps[3];
static uint8_t whoamI;
static uint8_t tx_buffer[1000];
static lsm6dsv16x_filt_settling_mask_t filt_settling_mask;

ai_handle network;
float aiInData[AI_NETWORK_IN_1_SIZE];
float aiOutData[AI_NETWORK_OUT_1_SIZE];
ai_u8 activations[AI_NETWORK_DATA_ACTIVATIONS_SIZE];
const char* activities[AI_NETWORK_OUT_1_SIZE] = {
    "bicep_curl", "hammer_curl", "wrist_rotation"};
ai_buffer * ai_input;
ai_buffer * ai_output;
```

Some function prototypes were defined for I2C read and write, send buffer over UART, IMU read loop and AI helper.

```

static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp,
                             uint16_t len);
static int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp,
                             uint16_t len);
static void tx_com( uint8_t *tx_buffer, uint16_t len );
static void platform_delay(uint32_t ms);

void lsm6dsv16x_read_data_polling(void);

static void AI_Init(void);
static void AI_Run(float *pIn, float *pOut);
static uint32_t argmax(const float * values, uint32_t len);

```

Inside the main loop, the AI module was initialized and the IMU data reading loop function “lsm6dsv16x_read_data_polling” was called which is the main part of the program for performing all the sensor data reading and running the AI library for motion classification.

```

AI_Init();
lsm6dsv16x_read_data_polling();

```

The function “lsm6dsv16x_read_data_polling” is the main function that handles sensor data reading, sending the data via neural network model to get an classification output and displaying the result through the UART. After waiting for the certain time dictated by “BOOT_TIME” macro, it checks the device ID and sets the output data rate and full scale for the accelerometer and gyroscope sensors. The output data rate for both sensors were kept at 15Hz and the full scale was set to 2g and 2000 dps for accelerometer and gyroscope respectively.

```

void lsm6dsv16x_read_data_polling(void)
{
    stmdev_ctx_t dev_ctx;
    /* Initialize mems driver interface */
    dev_ctx.write_reg = platform_write;
    dev_ctx.read_reg = platform_read;
    dev_ctx.mdelay = platform_delay;
    dev_ctx.handle = &SENSOR_BUS;

    /* Wait sensor boot time */
    platform_delay(BOOT_TIME);

    /* Check device ID */
    lsm6dsv16x_device_id_get(&dev_ctx, &whoamI);

    if (whoamI != LSM6DSV16X_ID)
        while (1);

    /* Restore default configuration */
    lsm6dsv16x_sw_por(&dev_ctx);

    /* Enable Block Data Update */
    lsm6dsv16x_block_data_update_set(&dev_ctx, PROPERTY_ENABLE);
    /* Set Output Data Rate.
     * Selected data rate have to be equal or greater with respect
     * with MLC data rate.
     */
    lsm6dsv16x_xl_data_rate_set(&dev_ctx, LSM6DSV16X_ODR_AT_15Hz);
    lsm6dsv16x_gy_data_rate_set(&dev_ctx, LSM6DSV16X_ODR_AT_15Hz);
    /* Set full scale */
    lsm6dsv16x_xl_full_scale_set(&dev_ctx, LSM6DSV16X_2g);
    lsm6dsv16x_gy_full_scale_set(&dev_ctx, LSM6DSV16X_2000dps);
}

```

This function contains the while loop which keeps on running the whole operation. It takes the raw accelerometer and gyro data and converts them to mg and mdps unit. “snprintf” was used to fill the buffer with the sensors reading and send it to UART to display in a terminal or use the readings for training the model.

```
while (1) {
    lsm6dsv16x_data_ready_t drdy;

    /* Read output only if new xl value is available */
    lsm6dsv16x_flag_data_ready_get(&dev_ctx, &drdy);

    if (drdy.drdy_xl) {
        /* Read acceleration field data */
        memset(data_raw_acceleration, 0x00, 3 * sizeof(int16_t));
        lsm6dsv16x_acceleration_raw_get(&dev_ctx, data_raw_acceleration);
        acceleration_mg[0] =
            lsm6dsv16x_from_fs2_to_mg(data_raw_acceleration[0]);
        acceleration_mg[1] =
            lsm6dsv16x_from_fs2_to_mg(data_raw_acceleration[1]);
        acceleration_mg[2] =
            lsm6dsv16x_from_fs2_to_mg(data_raw_acceleration[2]);
        //snprintf((char *)tx_buffer, sizeof(tx_buffer),
            // "%4.2f,%4.2f,%4.2f,",
            // acceleration_mg[0], acceleration_mg[1], acceleration_mg[2]);
        //tx_com(tx_buffer, strlen((char const *)tx_buffer));
    }

    /* Read output only if new xl value is available */
    if (drdy.drdy_gy) {
        /* Read angular rate field data */
        memset(data_raw_angular_rate, 0x00, 3 * sizeof(int16_t));
        lsm6dsv16x_angular_rate_raw_get(&dev_ctx, data_raw_angular_rate);
        angular_rate_mdps[0] =
            lsm6dsv16x_from_fs2000_to_mdps(data_raw_angular_rate[0]);
        angular_rate_mdps[1] =
            lsm6dsv16x_from_fs2000_to_mdps(data_raw_angular_rate[1]);
        angular_rate_mdps[2] =
            lsm6dsv16x_from_fs2000_to_mdps(data_raw_angular_rate[2]);
        //snprintf((char *)tx_buffer, sizeof(tx_buffer),
            // "%4.2f,%4.2f,%4.2f\r\n",
            // angular_rate_mdps[0], angular_rate_mdps[1],
            // angular_rate_mdps[2]);
        //tx_com(tx_buffer, strlen((char const *)tx_buffer));
    }
}
```

Thus obtained sensor data were normalized and accumulated in the input buffers as shown in code below.

```
aiInData[write_index + 0] = (float) acceleration_mg[0] / 2000.0f;
aiInData[write_index + 1] = (float) acceleration_mg[1] / 2000.0f;
aiInData[write_index + 2] = (float) acceleration_mg[2] / 2000.0f;
aiInData[write_index + 3] = (float) angular_rate_mdps[0] / 2000000.0f;
aiInData[write_index + 4] = (float) angular_rate_mdps[1] / 2000000.0f;
aiInData[write_index + 5] = (float) angular_rate_mdps[2] / 2000000.0f;
write_index += 6;
```

After the input buffer is filled with one input window, it runs the “AI_Run” function to obtain the output from neural network. It displays the probabilities for each output class and also provides the class with maximum probability by using function argmax().

```

if (write_index == AI_NETWORK_IN_1_SIZE) {
    write_index = 0;

    printf("Running inference\r\n");
    AI_Run(aiInData, aiOutData);

    /* Output results */
    for (uint32_t i = 0; i < AI_NETWORK_OUT_1_SIZE; i++) {
        printf("%8.6f ", aiOutData[i]);
    }
    uint32_t class = argmax(aiOutData, AI_NETWORK_OUT_1_SIZE);
    printf(": %d - %s\r\n", (int) class, activities[class]);
}

```

“platform_write” is the I2C write function that IMU driver calls whenever it wants to write into the register “reg”.

```

static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp,
                             uint16_t len)
{
    HAL_I2C_Mem_Write(handle, LSM6DSV16X_I2C_ADD_H, reg,
                      I2C_MEMADD_SIZE_8BIT, (uint8_t*) bufp, len, 1000);
    return 0;
}

```

Similarly, “platform_read” is the function called whenever IMU driver wants to read the data from the sensors.

```

static int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp,
                             uint16_t len)
{
    HAL_I2C_Mem_Read(handle, LSM6DSV16X_I2C_ADD_H, reg,
                     I2C_MEMADD_SIZE_8BIT, bufp, len, 1000);
    return 0;
}

```

“tx_com” function is used for UART transmission. It points to the buffer that needs to be transmitted along with the number of bytes to send denoted by the variable “len”. “platform_delay” function is just used for waiting for the specified time period.

```

static void tx_com(uint8_t *tx_buffer, uint16_t len)
{
    HAL_UART_Transmit(&huart2, tx_buffer, len, 1000);
}

static void platform_delay(uint32_t ms)
{
    HAL_Delay(ms);
}

```

“AI_init”, “AI_Run” and “argmax” are the three helper functions required for the machine learning tasks. “AI_init” is used for setting up the neural network once at the beginning. It handles the activation memory and initializes the network instance. It also deals with any error and gets the input and output buffer descriptions.

“AI_run” runs one forward pass of the neural network. It provides the input features in pIn and stores the output probabilities in pOut after the successful run. “ai_network_run” handles the forward pass operation.

```
static void AI_Init(void)
{
    ai_error err;

    /* Create a local array with the addresses of the activations buffers */
    const ai_handle act_addr[] = { activations };
    /* Create an instance of the model */
    err = ai_network_create_and_init(&network, act_addr, NULL);
    if (err.type != AI_ERROR_NONE) {
        int n = snprintf((char *)tx_buffer, sizeof(tx_buffer),
                        "ai_network_create error - type=%d code=%d\r\n",
                        err.type, err.code);
        tx_com(tx_buffer, n);
        Error_Handler();
    }
    ai_input = ai_network_inputs_get(network, NULL);
    ai_output = ai_network_outputs_get(network, NULL);
}

static void AI_Run(float *pIn, float *pOut)
{
    ai_i32 batch;
    ai_error err;

    /* Update IO handlers with the data payload */
    ai_input[0].data = AI_HANDLE_PTR(pIn);
    ai_output[0].data = AI_HANDLE_PTR(pOut);

    batch = ai_network_run(network, ai_input, ai_output);
    if (batch != 1) {
        err = ai_network_get_error(network);
        int n = snprintf((char *)tx_buffer, sizeof(tx_buffer),
                        "ai_network_create error - type=%d code=%d\r\n",
                        err.type, err.code);
        tx_com(tx_buffer, n);
        Error_Handler();
    }
}
```

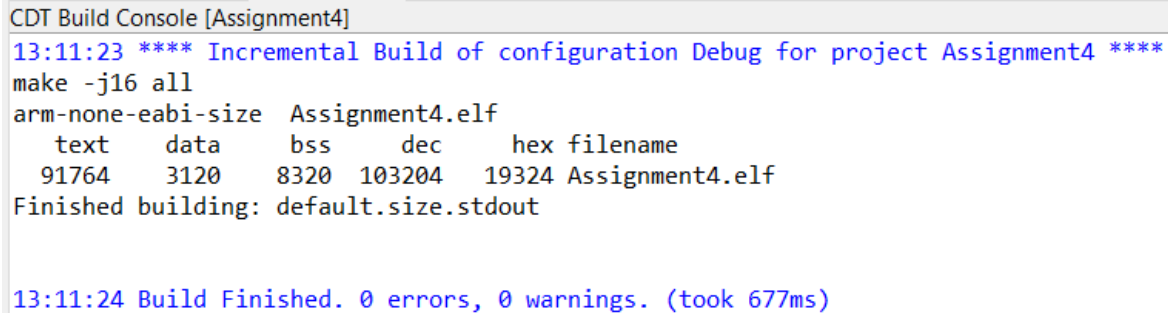
“argmax” function is used to find the largest output probabilities among the 3 output classes. It then returns the index of the most probable class which is displayed via UART.

```
static uint32_t argmax(const float * values, uint32_t len)
{
    float max_value = values[0];
    uint32_t max_index = 0;
    for (uint32_t i = 1; i < len; i++) {
        if (values[i] > max_value) {
            max_value = values[i];
            max_index = i;
        }
    }
    return max_index;
}
```

For printf (used in earlier code) to print via UART, “_write” function is required which redirects all the printf operation to use UART for displaying output.

```
/* Redirect printf to USART */
int _write(int file, char *ptr, int len)
{
    /* Send all characters over UART */
    HAL_UART_Transmit(&huart2, (uint8_t *)ptr, len, HAL_MAX_DELAY);
    return len;
}
```

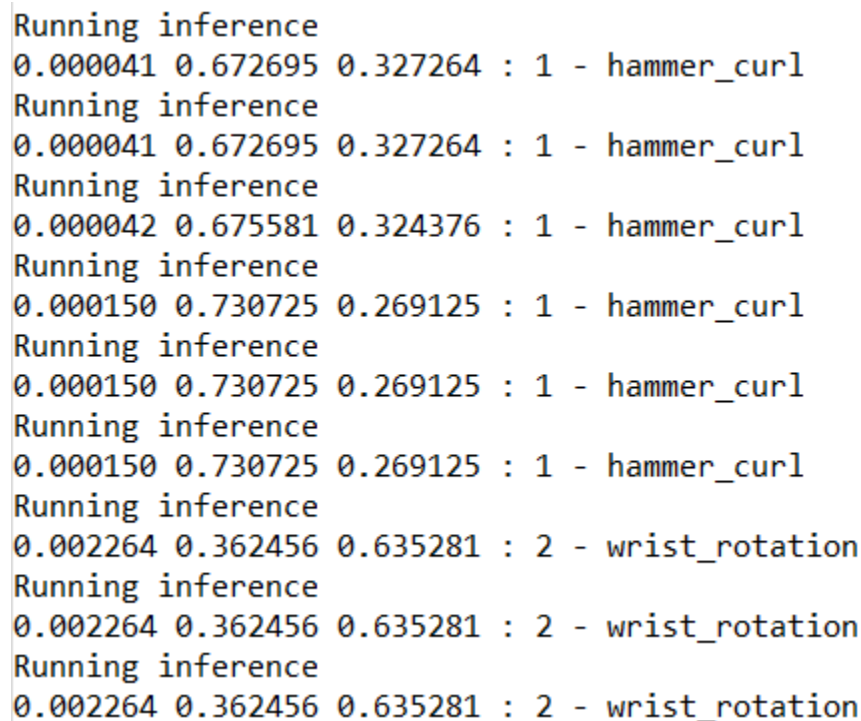
Build and output



```
CDT Build Console [Assignment4]
13:11:23 **** Incremental Build of configuration Debug for project Assignment4 ****
make -j16 all
arm-none-eabi-size Assignment4.elf
   text    data     bss     dec     hex filename
  91764    3120     8320  103204   19324 Assignment4.elf
Finished building: default.size.stdout

13:11:24 Build Finished. 0 errors, 0 warnings. (took 677ms)
```

Figure 2: Screenshot of successful build of the code



```
Running inference
0.000041 0.672695 0.327264 : 1 - hammer_curl
Running inference
0.000041 0.672695 0.327264 : 1 - hammer_curl
Running inference
0.000042 0.675581 0.324376 : 1 - hammer_curl
Running inference
0.000150 0.730725 0.269125 : 1 - hammer_curl
Running inference
0.000150 0.730725 0.269125 : 1 - hammer_curl
Running inference
0.000150 0.730725 0.269125 : 1 - hammer_curl
Running inference
0.002264 0.362456 0.635281 : 2 - wrist_rotation
Running inference
0.002264 0.362456 0.635281 : 2 - wrist_rotation
Running inference
0.002264 0.362456 0.635281 : 2 - wrist_rotation
```

Figure 3: Screenshot of output

Figure 2 and 3 shows the successful build of the code and display of output in the terminal via UART. The output displays the probability of each classes along with the class with the highest probability as seen in Figure 3.

Python script for model training

Python list for storing the input features and output labels were created and each csv file corresponding to each class data was loaded and the input features and output labels were stored in the corresponding list as seen in the code below.

```
x_recordings = []
y_recordings = []

# labels for different classes of motion
labels = ['bicep_curl', 'hammer_curl', 'wrist_rotation']
base_dir = Path(__file__).resolve().parent # To get the base directory of this python script

for i, label in enumerate(labels):
    # iterates over each files corresponding to each class
    filename = base_dir / f"{label}.csv"
    data = np.loadtxt(filename, delimiter=',', skiprows=1)
    features = data[:, 0:6] # first 6 columns are the input features
    x_recordings.append(features)
    y_recordings.append(i)
```

Data frames were then created to obtain the batch of data representing the time sequence data up to certain period. Since, in order to recognized certain gesture/motion, a time sequence data is required. Hence, the function frame() was created to create a fixed size input to feed into the neural network.

```
# function to create frames from the data
def frame(x, frame_len, hop_len):

    assert x.shape == (len(x), 6)
    assert x.shape[0] >= frame_len
    assert hop_len >= 1

    n_frames = 1 + (x.shape[0] - frame_len) // hop_len
    shape = (n_frames, frame_len, x.shape[1])
    strides = ((hop_len * x.strides[0],) + x.strides)
    return np.lib.stride_tricks.as_strided(x, shape = shape, strides =      strides)

x_frames = []
y_frames = []

# creating frame overlapping
for i, recording in enumerate(x_recordings):
    frames = frame(recording, 26, 13) # 50% overlap
    x_frames.append(frames)
    y_frames.append(np.full(frames.shape[0], y_recordings[i]))
```

```
x_frames = np.concatenate(x_frames)
y_frames = np.concatenate(y_frames)
print("x_frames shape:", x_frames.shape) # (total_frames, 26, 6)
print("y_frames shape:", y_frames.shape) # (total_frames,)
```

The data obtained from the csv files were then normalized using full scale range of each sensor data. Thus obtained normalized data were then split into train/validation/test set in the 60/20/20 ratio.

```
# Data normalization
acc = x_frames[:, :, 0:3] # ax, ay, az in mg
gyro = x_frames[:, :, 3:6] # gx, gy, gz in mdps

acc_norm = acc / 2000.0 # full-scale is 2g
gyro_norm = gyro / 2000000.0 # full-scale is 2000dps

x_frames_norm = np.concatenate([acc_norm, gyro_norm], axis = -1)

# train/val/test dataset split at 60/20/20 ratio
x_train, x_temp, y_train, y_temp = train_test_split(
    x_frames_norm, y_frames, test_size = 0.4, random_state = 22, stratify = y_frames
)

x_val, x_test, y_val, y_test = train_test_split(
    x_temp, y_temp, test_size = 0.50, random_state = 22, stratify = y_temp
)
```

In order to obtain the good model, 3 different configurations of hyperparameters were tried as listed below.

```
# Hyperparameter grid to try
hyperparams_list = [
    {"filters1": 16, "filters2": 8, "dense_units": 64, "dropout": 0.5},
    {"filters1": 32, "filters2": 16, "dense_units": 64, "dropout": 0.5},
    {"filters1": 16, "filters2": 8, "dense_units": 128, "dropout": 0.5},
]
```

Using different hyperparameters combination, each model was defined using the `build_model()` function. Two 1D convolutional layers followed by dropout and fully connected layer was used as the model architecture. Adam optimizer was used for optimization and sparse categorical cross entropy was used as the loss function. The hyperparameters were chosen based on the highest accuracy.

```
# function to build model based on provided hyperparameters value
def build_model(filters1, filters2, dense_units, dropout):
    model = tf.keras.models.Sequential([
        tf.keras.layers.Conv1D(filters=filters1, kernel_size=3, activation='relu',
                                input_shape=(26, 6)),
```

```

        tf.keras.layers.Conv1D(filters=filters2, kernel_size=3, activation='relu'),
        tf.keras.layers.Dropout(dropout),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(dense_units, activation='relu'),
        tf.keras.layers.Dense(3, activation='softmax')
    ])
    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model

```

Each model was trained upto 40 epochs. The validation dataset's accuracy was used for choosing the best model with the highest accuracy.

```

num_epochs = 40

for idx, hp in enumerate(hyperparams_list):
    print(f"Training config {idx+1}/{len(hyperparams_list)}: {hp}")
    model = build_model(**hp)
    history = model.fit(x_train, y_train, epochs = num_epochs, validation_data = (x_val,
y_val), verbose = 1)
    # Use best validation accuracy of this run
    val_acc = max(history.history["val_accuracy"])
    print(f"Config {hp} best val_accuracy = {val_acc:.4f}")

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model = model
        best_history = history
        best_config = hp

print("\nBest hyperparameters based on validation accuracy:")
print(best_config)
print(f"Best val_accuracy: {best_val_acc:.4f}")

```

The best model was then tested using the test dataset. The loss and accuracy of the model with test dataset was then displayed. The best configuration was with filter1 = 16, filter2 = 8, dense unit = 64 and dropout = 0.5 as seen in Figure 4. The test accuracy with this model was 97.6%.

```

# Evaluate best model on test set
test_loss, test_acc = best_model.evaluate(x_test, y_test, verbose = 2)
print("Test loss:", test_loss)
print("Test accuracy:", test_acc)

```

```

Best hyperparameters based on validation accuracy:
{'filters1': 16, 'filters2': 8, 'dense_units': 64, 'dropout': 0.5}
Best val_accuracy: 0.9524
2/2 - 0s - 21ms/step - accuracy: 0.9762 - loss: 0.1581
Test loss: 0.15808726847171783
Test accuracy: 0.976190447807312

```

Figure 4: Test loss and Accuracy of the trained model

Figure 5 and 6 shows the loss and accuracy vs epochs for training and validation dataset when the model with best hyperparameters configuration was used. It displays the successful learning of the model to distinguish the three motion classes. Both the training and validation loss decreases in a similar way till about 15 epochs and the training loss continues to decrease more than the validation loss afterward. The validation loss also has decreasing trend after 15th epochs so there is no problem of overfitting in this case.

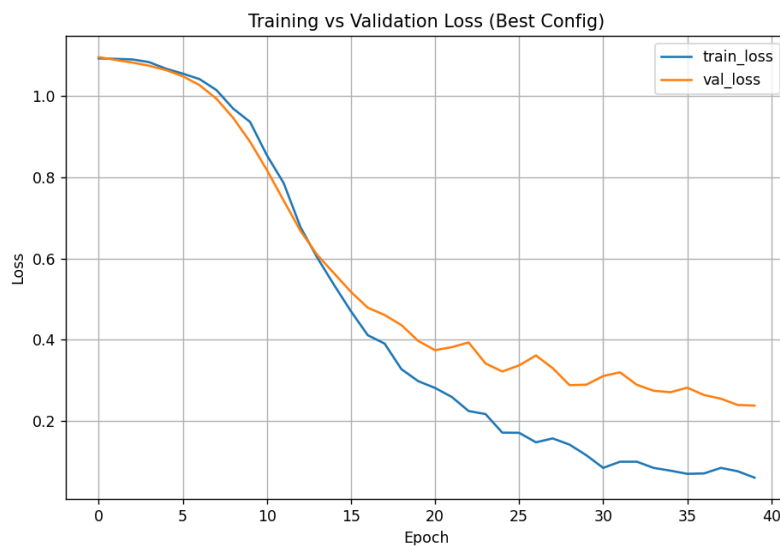


Figure 5: Loss with training and validation dataset for the best hyperparameter config

Similar result is seen in the accuracy curves in Figure 6. Both accuracies reach almost 90% before the 25th epochs. The validation accuracy is totally flat after 30 epochs and this indicates that the 30 epochs are enough to obtain the same accuracy as 40 epochs. The high accuracy in validation set also indicates the ability of the model to generalize to the unknown dataset.

Figure 7 shows the confusion matrix for the test dataset using the best hyperparameter configuration. It demonstrates that the model performs very well in classifying the three motion classes. Bicep curl and hammer curl has perfect classification where the model classified all 14 windows of them correctly. Only one data window of wrist rotation was incorrectly classified as the bicep curl. This indicates that the temporal features learned by neural network model with the overlapping windows of 26 samples give the good result in classifying the 3 motion patterns.

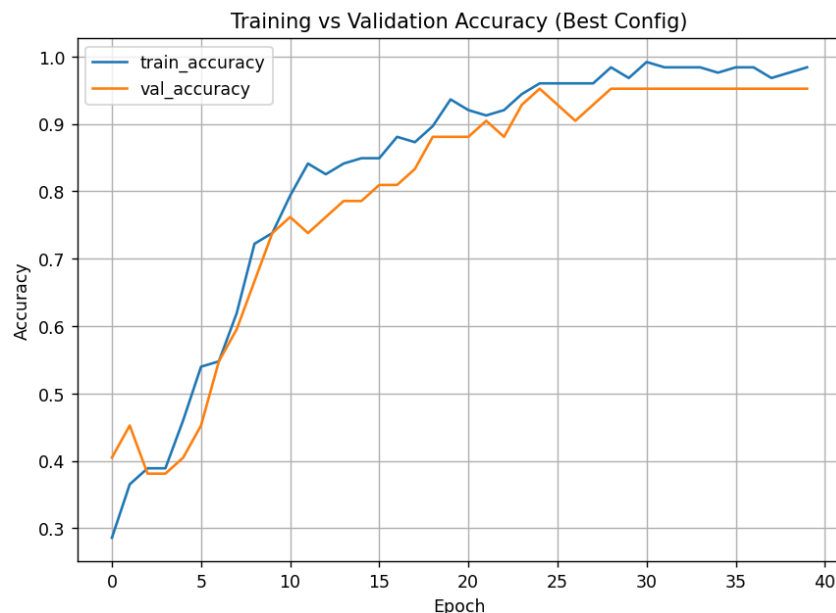


Figure 6: Training and validation accuracy for best hyperparameter configuration

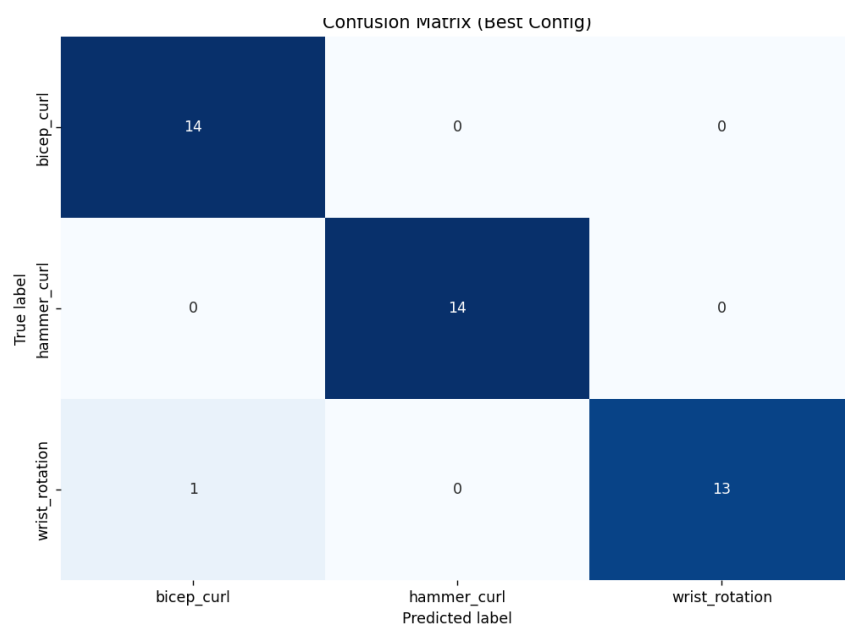


Figure 7: Confusion matrix with test dataset

Figure 8 shows the ROC curves for each motion classes within the test dataset. The hammer curl achieves the perfect AUC of 1.00 indicating the perfect classification of hammer curl across all possible decision thresholds. The bicep curl and wrist rotation classification also have high performance with AUC equal to 0.96 and 0.94 respectively. These ROC curves indicate that the model is able to keep the true positive rate high while keeping the false positive rate low.

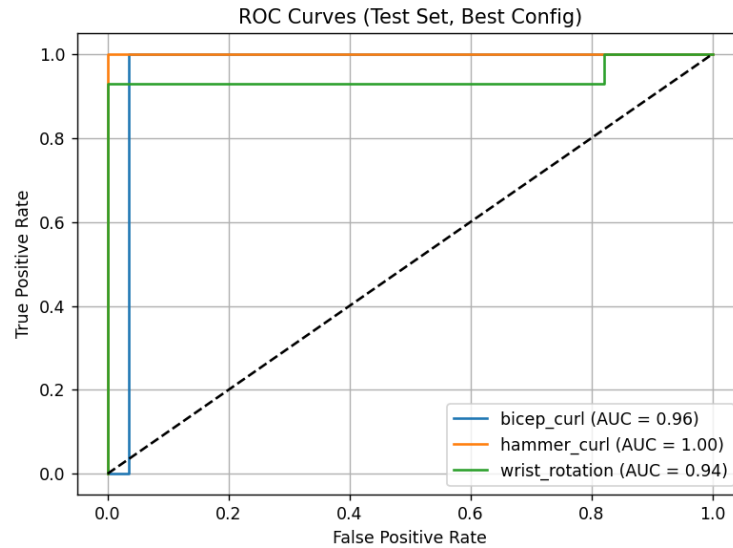


Figure 8: ROC curve for test dataset

In conclusion, it can be said that the neural network model designed using 1D CNN is suitable for classifying the three motion patterns in real-time. All the parameters and curves show that the model provides very good classification with proper generalization and higher reliability.

3. Declaration

I understand the Student Academic Misconduct Policy -
<http://studentconduct.unlv.edu/misconduct/policy.html>

"This assignment submission is my own, original work".

Chetan Dahal